Uganda Christian University
Department of Electronics, Communication and Computer Engineering

Communication Theory

Practical 8 (Mini-Project): End-to-End Communication System Simulation over an ARINC 429 Link

Name	**Registration number**
Shiisa Jacob	S25B48/056
Kasozi Nimusiima Erina	S25B48/046

Language: Python (NumPy, SciPy, Matplotlib)

October 2026

Abstract

This report describes a simulation of a complete digital communication chain:

source → sampling → PCM → line coding → BPSK modulation → AWGN channel → demodulation → decoding → reconstruction.

The chain is set in an aviation context. A 300 Hz sensor signal is sampled at 8 kHz and quantized to 8 bits. The samples are packed into ARINC 429 words and sent with the ARINC 429 bipolar return-to-zero line code.

With a good channel, the signal is recovered to within the quantization error (MSE 6.34×10^{-6}, SQNR 49.0 dB). The simulated bit error rate matches BPSK theory. Below about 9.9 dB E_b/N_0, channel errors rather than quantization set the reconstruction error, which follows $\mathrm{MSE}_q + \frac{4}{3}\,\mathrm{BER}$. Shannon's theorem shows the same bit rate could be carried error-free down to −1.21 dB. The gap between −1.21 dB and 9.9 dB exists because ARINC 429 parity can only detect errors, not correct them.

Contents

1 Objective **2**

2 Background: how ARINC 429 operates **2**

3 System design **2**
3.1 Block diagram . . . 2
3.2 Parameters . . . 3
3.3 Software structure . . . 3

4 Results at each stage **3**
4.1 Source and sampling . . . 3
4.2 Quantization and PCM . . . 4
4.3 Line coding: ARINC 429 framing and bipolar RZ . . . 5
4.4 Digital modulation: BPSK . . . 5
4.5 AWGN channel . . . 6
4.6 Demodulation . . . 6
4.7 Decoding and reconstruction . . . 7
4.8 Parity check example . . . 8

5 Performance analysis **8**
5.1 Bit errors and word checks . . . 8
5.2 Reconstruction MSE against channel SNR . . . 9

6 Discussion: link to Shannon's channel capacity theorem **10**

7 Conclusions **11**

8 Limitations **11**

A Code **11**

1 Objective

The practical asks for a complete simulated chain, with plots at every stage, a study of how channel SNR affects the quality of the reconstructed signal, and a discussion linking the results to Shannon's channel capacity theorem.

In this work the chain is applied to the ARINC 429 avionics data bus. The message represents an analog sensor on an aircraft, such as an engine vibration pickup. A data concentrator samples it and sends the values as ARINC 429 words.

2 Background: how ARINC 429 operates

ARINC 429 is the data bus used on most commercial aircraft to move sensor and system data between avionics units. Table 1 lists the parts that matter for this simulation.

Table 1: ARINC 429 features and how they are modelled.

	ARINC 429	**This simulation**
Topology	One transmitter, up to 20 receivers, one direction	One transmitter, one receiver
Medium	Shielded twisted pair, differential	AWGN channel
Line code	Bipolar RZ: HI = +10 V, NULL = 0 V, LO = −10 V	Same, normalised to ±1
Bit rate	12.5–14.5 kbit/s (low speed), 100 kbit/s (high speed)	100 kbit/s for the throughput check
Word	32 bits: label (1–8), SDI (9–10), data (11–29), SSM (30–31), parity (32)	Same layout; data field holds two 8-bit samples
Word gap	At least 4 bit times of NULL	4 bit times
Error control	Odd parity: detects, does not correct	Parity, plus label and SSM checks

ARINC 429 is a baseband bus and has no carrier. The practical requires a digital modulation stage, so the ARINC bit stream is modelled as being relayed over a BPSK radio link. Flight-test telemetry does this when it sends recorded ARINC 429 bus traffic to a ground station. The plain baseband bus is also run through the same channel for comparison (Section 5.1).

3 System design

3.1 Block diagram

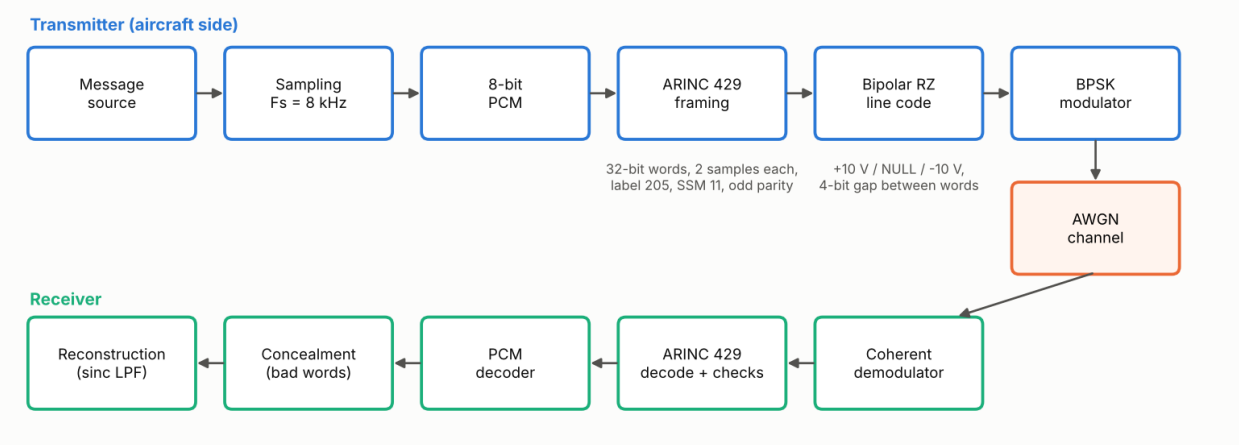


Figure 1: Complete system. Transmitter stages on top, receiver stages below.

3.2 Parameters

Table 2: Simulation parameters.

Parameter	Value
Message	$m(t) = \sin(2\pi \cdot 300\,t)$, 50 ms
Sampling rate F_s	8 kHz (401 samples)
Quantizer	8-bit uniform mid-rise over $[-1, 1]$, $\Delta = 2/256$
ARINC label / SDI / SSM	205 (octal) / 00 / 11
Samples per word	2 (201 words, one padding sample)
Gap between words	4 bit times
Simulation samples per bit	40
Carrier	4 cycles per bit time
Monte Carlo	150 runs per E_b/N_0 point (964 800 bits)

3.3 Software structure

The project is written in Python. `main.py` runs the whole chain, saves every figure, and writes the sweep results to `figures/results.csv`. Each stage is a separate module in `functions/`: `source`, `pcm`, `arinc429`, `line_coding`, `modulation`, `channel`, `reconstruction`, `metrics` and `system`. Appendix A lists the key code.

4 Results at each stage

4.1 Source and sampling

The 300 Hz sine is sampled at 8 kHz. The Nyquist rate is 600 Hz, so sampling at 8 kHz is about 13 times the minimum and there is no aliasing.

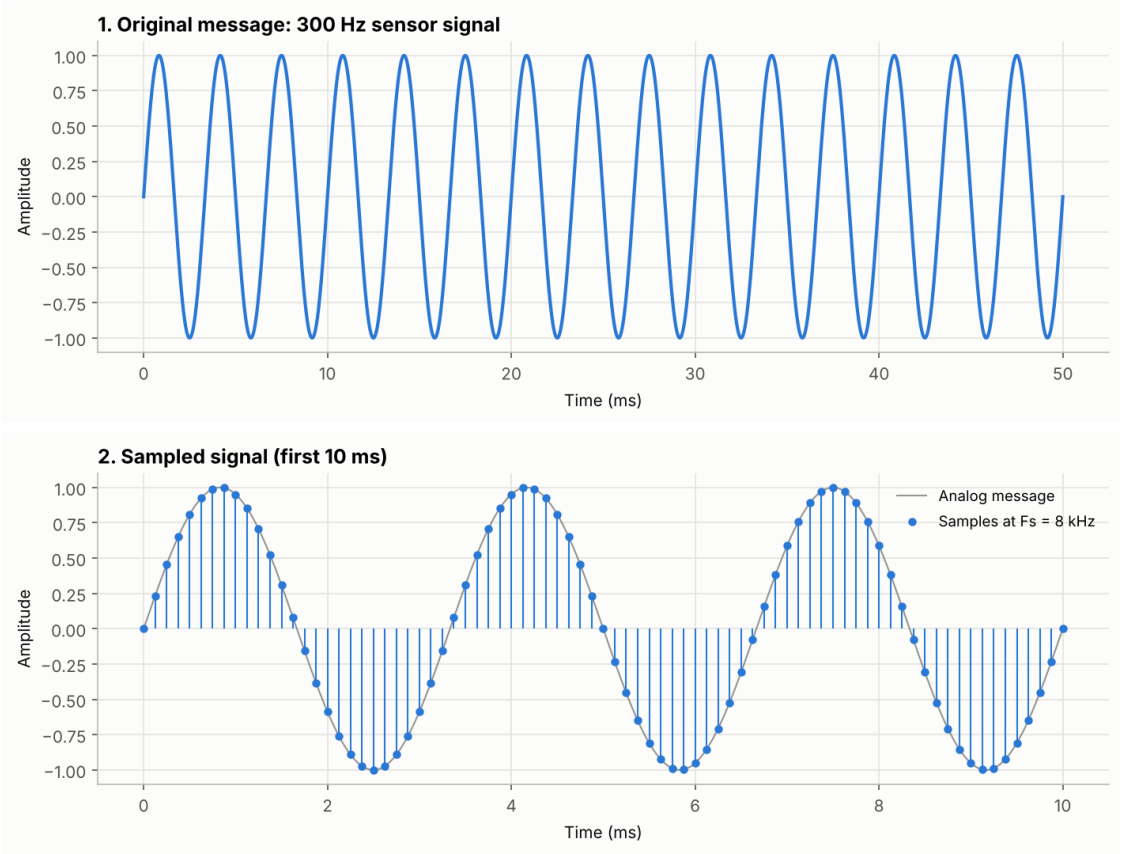


Figure 2: Original message (top) and sampled signal over the first 10 ms (bottom).

4.2 Quantization and PCM

The quantizer maps each sample to one of 256 levels. The error stays within $\pm\Delta/2$ (Figure 3). Each code is written as 8 bits, MSB first. The first codes are 128, 157, 186, 211, 231.

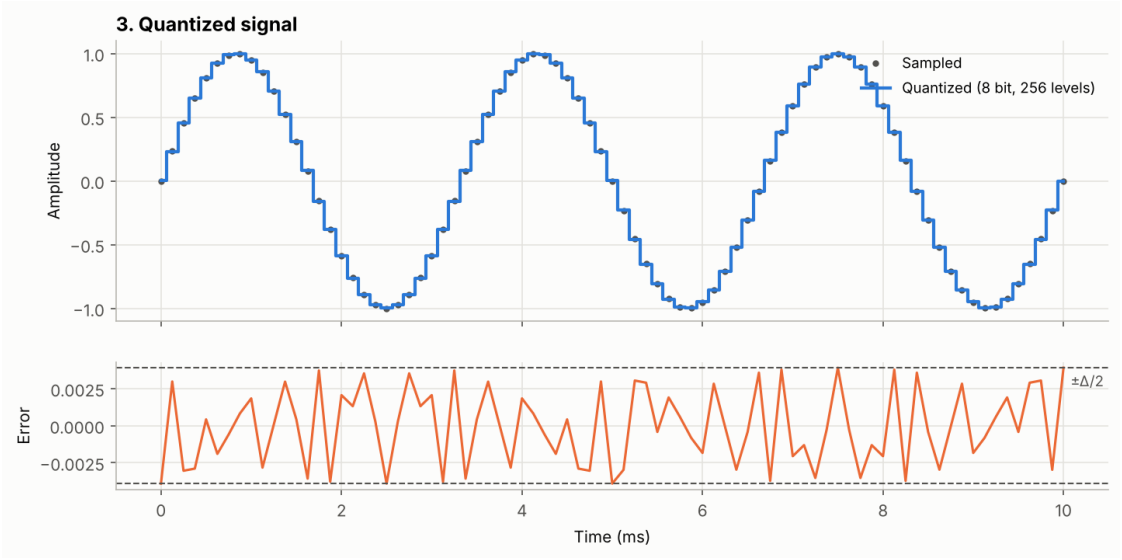


Figure 3: Quantized signal and quantization error.

The measured quantization MSE is 6.34×10^{-6}, against $\Delta^2/12 = 5.09 \times 10^{-6}$. The difference comes from the signal itself. A 300 Hz tone sampled at 8 kHz repeats every 80 samples, so the error

takes only 80 distinct values and is not exactly uniform. The SQNR is

$$\mathrm{SQNR} = 10\log_{10}\frac{P_m}{\mathrm{MSE}_q} = 49.0\,\mathrm{dB},$$

within 1 dB of the rule $6.02n + 1.76 = 49.9\,\mathrm{dB}$.

4.3 Line coding: ARINC 429 framing and bipolar RZ

Each 32-bit word carries two samples (Figure 4). The label is sent MSB first, as ARINC 429 requires. The rest of the word is sent LSB first. Bits 27–29 are unused and set to zero. Bit 32 is set so the word has an odd number of ones.

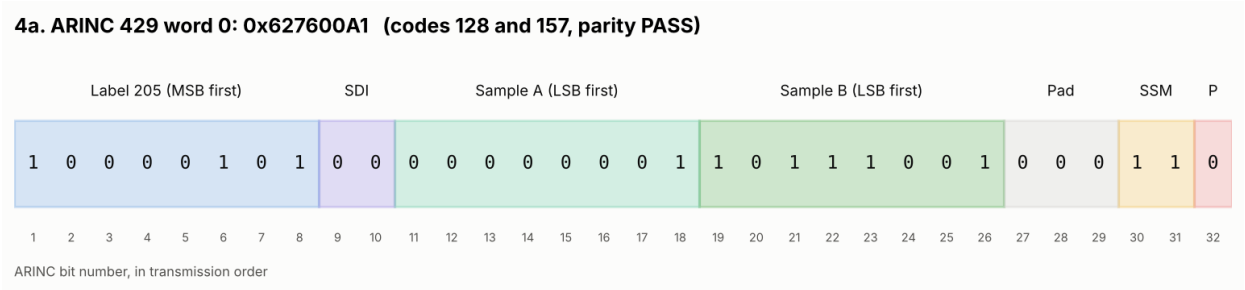


Figure 4: First ARINC 429 word, `0x627600A1`, carrying codes 128 and 157.

The bits are then sent with the ARINC 429 bipolar RZ code (Figure 5). A 1 is a 10 V pulse and a 0 is a −10 V pulse, each lasting the first half of the bit time and followed by NULL. Receivers use the 4-bit NULL gap between words to find the start of each word.

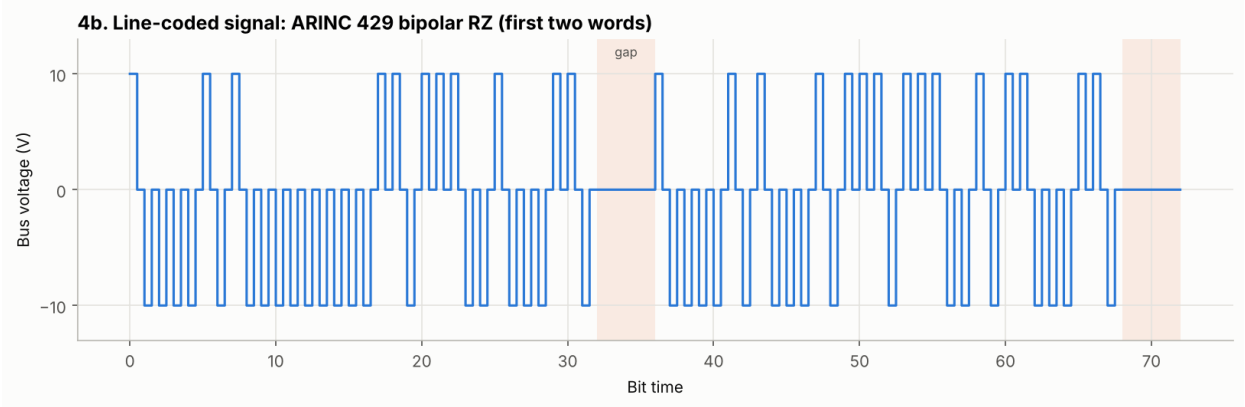


Figure 5: Bipolar RZ line code for the first two words. Shaded: inter-word gaps.

4.4 Digital modulation: BPSK

The RZ waveform multiplies a carrier with 4 cycles per bit time:

$$s(t) = b(t)\cos(2\pi f_c t), \qquad b(t) \in \{+1, 0, -1\}.$$

A 1 is a half-bit carrier burst at phase 0 and a 0 is a burst at phase π. This is BPSK with half-bit pulses.

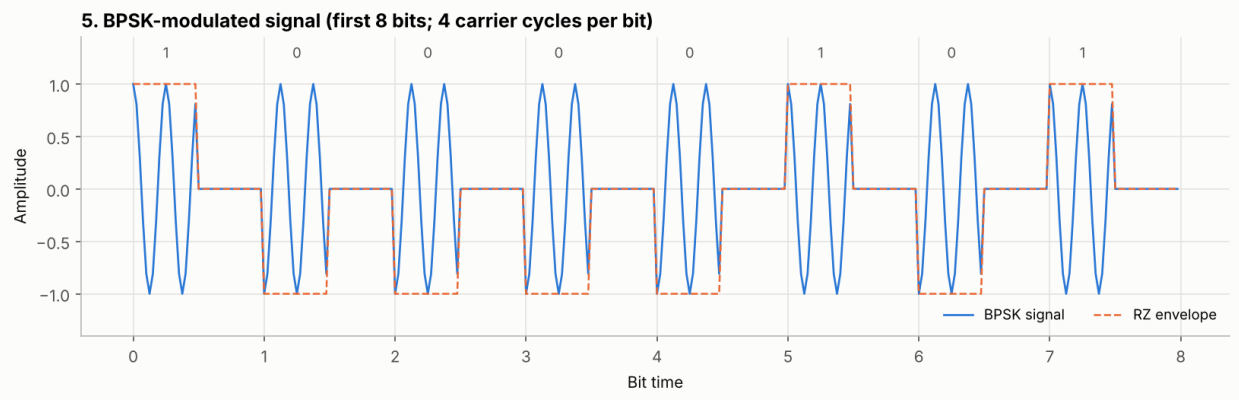


Figure 6: BPSK signal for the first 8 bits, with the RZ envelope dashed.

4.5 AWGN channel

White Gaussian noise with variance $\sigma^2 = N_0/2$ is added. The noise level is set by E_b/N_0, where E_b is the transmitted energy per bit. E_b/N_0 is used because BER theory and the Shannon limit are both written in terms of it.

The practical's template uses `awgn(x, SNR, 'measured')`, which sets the SNR per sample. For this waveform the two are related by a fixed offset:

$$\mathrm{SNR}_{\mathrm{sample}} = \frac{\overline{s^2}}{\sigma^2} = 2\frac{E_b}{N_0} \cdot \frac{N_{\mathrm{bits}}}{N_{\mathrm{samples}}} \quad \Rightarrow \quad \mathrm{SNR}_{\mathrm{sample}}\,[\mathrm{dB}] = E_b/N_0\,[\mathrm{dB}] - 13.52.$$

The offset is large for two reasons. The receiver adds up 20 samples per bit, and the NULL halves and gaps add samples that carry no signal. So a per-sample SNR of 0 dB is already $E_b/N_0 \approx 13.5$ dB, which is error-free for BPSK. Sweeping only 0–25 dB of per-sample SNR therefore gives a flat MSE curve and shows nothing.

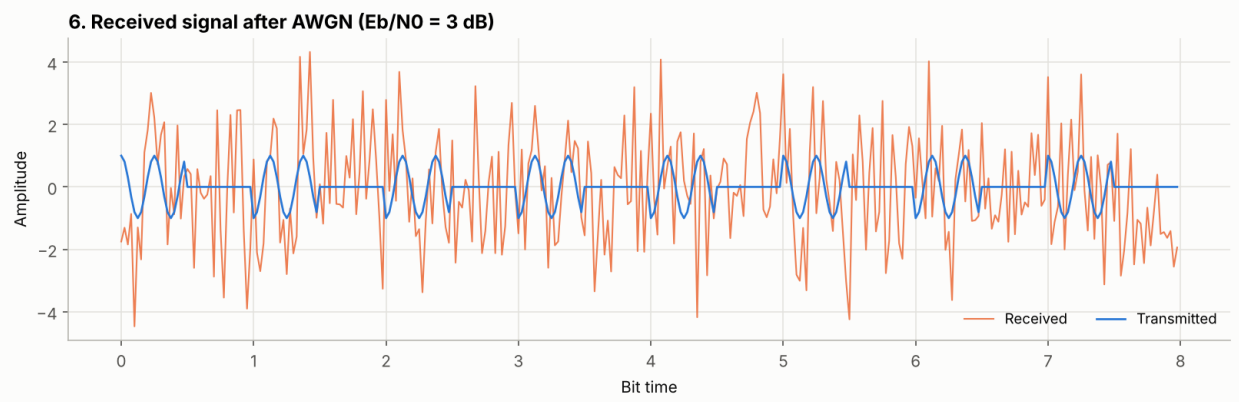


Figure 7: Received signal at $E_b/N_0 = 3$ dB against the transmitted signal.

4.6 Demodulation

The receiver multiplies the received signal by the carrier and sums over the active half of each bit:

$$z_k = \sum_{n \in \text{active half}} r[n]\cos(2\pi f_c n T_s), \qquad \hat{b}_k = \begin{cases} 1 & z_k > 0 \\ 0 & z_k \le 0 \end{cases}$$

This is the matched filter for the half-bit pulse. Figure 8 shows the decision values at 3 dB, where a few values fall on the wrong side of zero. Over the whole run, 148 of 6432 bits were in error (BER $= 2.30 \times 10^{-2}$, against 2.29×10^{-2} from theory).

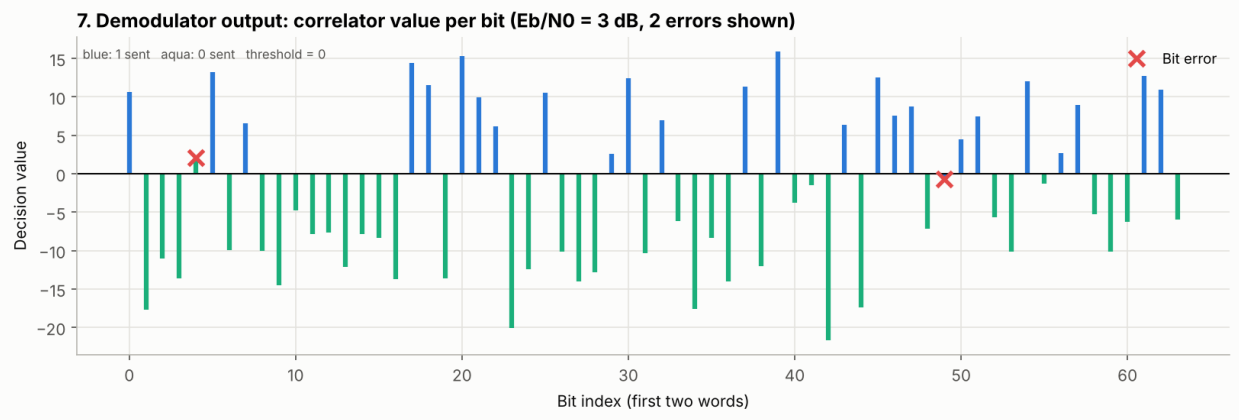


Figure 8: Correlator output per bit for the first two words at $E_b/N_0 = 3\,\mathrm{dB}$.

4.7 Decoding and reconstruction

The receiver groups the bits back into 32-bit words. It accepts a word only if parity is odd, the label is 205 and the SSM is 11. The 8-bit codes are mapped back to amplitude levels, and the waveform is rebuilt with ideal low-pass (sinc) interpolation:

$$\hat{m}(t) = \sum_n \hat{m}[n]\ \mathrm{sinc}(F_s t - n).$$

Three ways of handling rejected words are compared:

- **Raw decode:** use every word as received, with no checks.
- **Parity + hold:** replace a rejected word with the last good sample. A basic avionics receiver does this when it drops a bad word.
- **Parity + interpolation:** replace a rejected word by interpolating linearly between the good samples on either side.

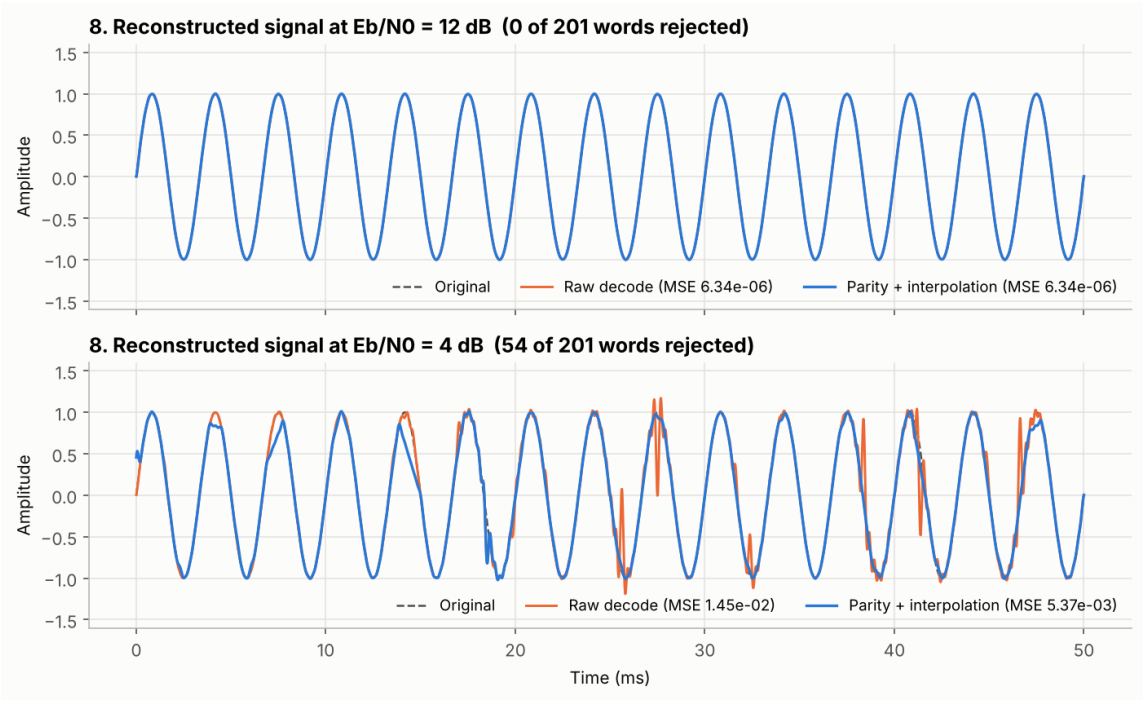


Figure 9: Reconstructed signal at $E_b/N_0 = 12\,\mathrm{dB}$ (no words rejected) and $4\,\mathrm{dB}$ (54 of 201 words rejected).

4.8 Parity check example

Flipping bits in a clean word (codes 231 and 246) shows what parity can and cannot catch:

Bits flipped	Parity	Decoded codes
none	PASS	231, 246
15	FAIL	247, 246
15 and 21	PASS	247, 242 (wrong, not detected)

5 Performance analysis

5.1 Bit errors and word checks

Figure 10 (left) shows that the simulated BER follows the BPSK result

$$P_b = \tfrac{1}{2}\operatorname{erfc}\sqrt{E_b/N_0}$$

to within Monte Carlo scatter. The baseband ARINC bus gives the same BER at the same E_b/N_0. Bipolar signalling with a matched filter is antipodal, the same as BPSK, so the carrier changes the spectrum but not the error rate.

The fraction of rejected words matches the probability that a word has at least one bit error, $1-(1-P_b)^{32}$. Parity on its own misses words with an even number of errors:

$$P_{\text{undetected}} = \tfrac{1}{2}\left[1+(1-2P_b)^{32}\right] - (1-P_b)^{32}.$$

The simulated rate of bad words accepted is lower than this, because the label and SSM checks catch some of them. At high error rates, though, many bad words still get through: at 0 dB, 15.6% of all words are bad but accepted.

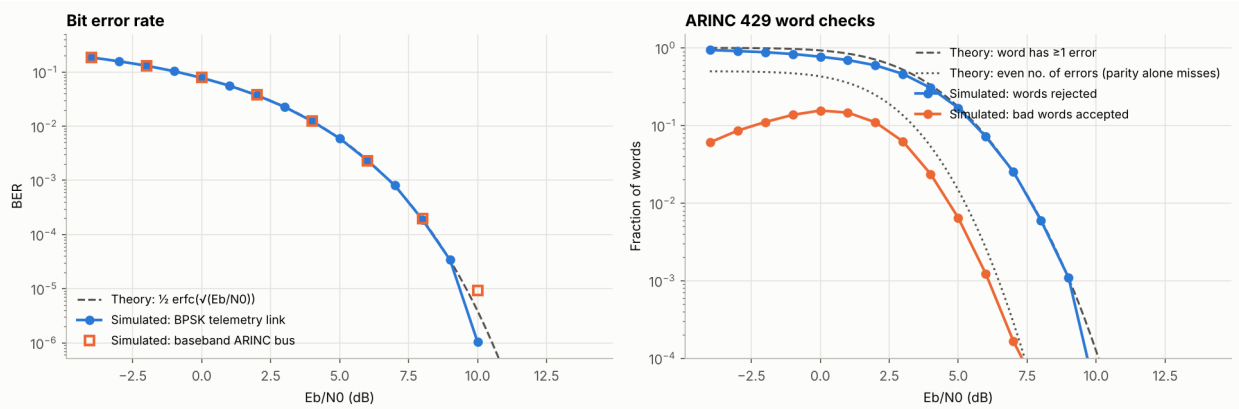


Figure 10: Left: BER against E_b/N_0. Right: rejected words and bad words that pass all checks.

5.2 Reconstruction MSE against channel SNR

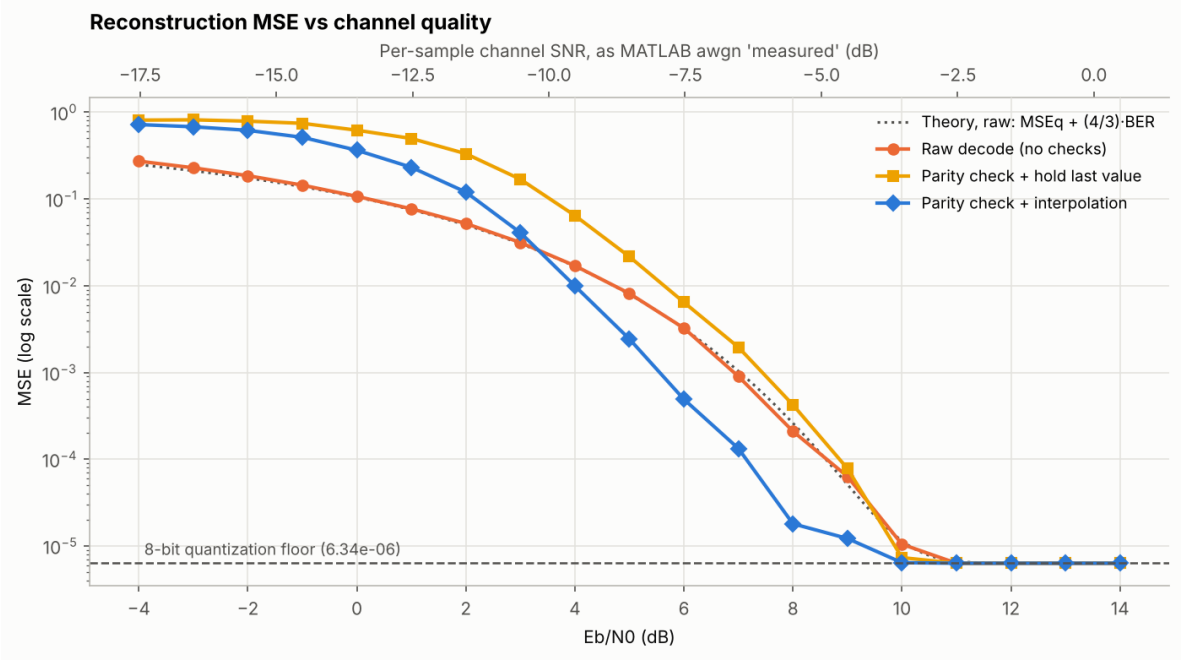


Figure 11: Reconstruction MSE against E_b/N_0 (bottom axis) and per-sample SNR (top axis).

Table 3: Selected results (150 runs per point).

E_b/N_0 (dB)	SNR/sample (dB)	BER	BER theory	MSE raw	MSE hold	MSE interp.
0	−13.5	7.81×10^{-2}	7.86×10^{-2}	1.07×10^{-1}	6.20×10^{-1}	3.64×10^{-1}
2	−11.5	3.73×10^{-2}	3.75×10^{-2}	5.21×10^{-2}	3.32×10^{-1}	1.20×10^{-1}
4	−9.5	1.24×10^{-2}	1.25×10^{-2}	1.70×10^{-2}	6.44×10^{-2}	9.94×10^{-3}
6	−7.5	2.35×10^{-3}	2.39×10^{-3}	3.25×10^{-3}	6.44×10^{-3}	4.91×10^{-4}
8	−5.5	1.89×10^{-4}	1.91×10^{-4}	2.10×10^{-4}	4.24×10^{-4}	1.81×10^{-5}
10	−3.5	1.04×10^{-6}	3.87×10^{-6}	1.05×10^{-5}	7.35×10^{-6}	6.43×10^{-6}
12	−1.5	0	9.01×10^{-9}	6.34×10^{-6}	6.34×10^{-6}	6.34×10^{-6}

Effect of channel SNR. Above about 10 dB E_b/N_0 (−3.6 dB per-sample SNR), bit errors are rare and the MSE stays at the 8-bit quantization floor, 6.34×10^{-6}. More SNR does not improve it. Below that point the MSE rises quickly.

The raw decoder's MSE can be predicted. An error in bit k of an n-bit code shifts the sample by $2^k\Delta$. With independent errors at rate P_b, the expected squared error per sample is

$$\sum_{k=0}^{n-1} P_b\,(2^k\Delta)^2 = P_b\,\Delta^2\,\frac{4^n-1}{3} \approx \tfrac{4}{3}P_b \qquad (\Delta = 2/2^n),$$

so

$$\mathrm{MSE}_{\mathrm{raw}} \approx \mathrm{MSE}_q + \tfrac{4}{3}\,P_b.$$

This curve is the dotted line in Figure 11 and agrees with the simulation. Errors in the most significant bit dominate. The channel contribution equals the quantization contribution when $\frac{4}{3}P_b = \mathrm{MSE}_q$, which happens at $E_b/N_0 = 9.9$ dB.

Effect of the parity check. Parity only helps if the receiver has something better to put in place of a rejected word.

- **Holding the last good value is worse than doing nothing over most of the range.** Most rejected words are wrong only in a low-order bit, which costs little. Holding throws away two good samples of a sine that can change by up to 0.24 per sample.
- **Interpolation does better from about 4 dB upward.** It reduces the MSE about 7 times at 6 dB and 12 times at 8 dB. Below about 3 dB most words are rejected, the gaps become long, and interpolation does worse than the raw decoder.

With detection-only checks, as in ARINC 429, the choice of concealment method makes as much difference as a few dB of SNR.

6 Discussion: link to Shannon's channel capacity theorem

Shannon's theorem gives the highest rate that a band-limited AWGN channel can carry with an arbitrarily small error probability:

$$C = B \log_2\left(1 + \frac{S}{N}\right), \qquad \frac{S}{N} = \frac{E_b}{N_0} \cdot \frac{R_b}{B}.$$

Setting $C = R_b$ gives the smallest E_b/N_0 at which the bit rate R_b can be carried at all:

$$\left(\frac{E_b}{N_0}\right)_{\min} = \frac{2^{R_b/B} - 1}{R_b/B}.$$

With half-bit pulses, the BPSK signal occupies about $B = 4R_b$ between spectral nulls, which gives -1.21 dB. As $B \to \infty$ this falls to $\ln 2 = -1.59$ dB, the Shannon limit.

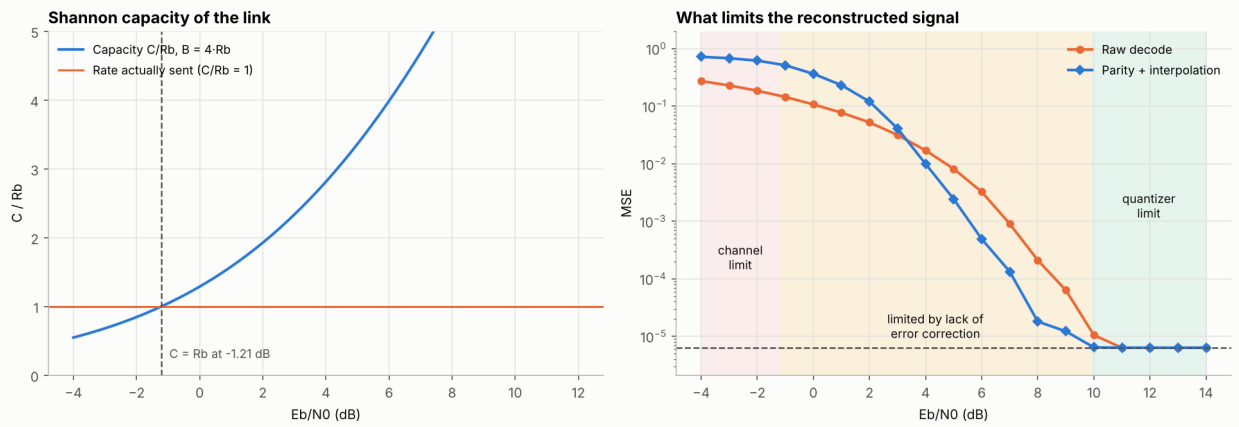


Figure 12: Left: capacity relative to the bit rate. Right: the three regions of the MSE curve.

This splits the MSE curve into three regions (Figure 12):

1. **Below -1.2 dB: the channel is the bottleneck.** Capacity is less than the bit rate, so no modulation or coding scheme can deliver the samples without errors. The only options are a lower bit rate (fewer bits per sample or a lower F_s) or more transmit power.
2. **From -1.2 dB to about 10 dB: the design is the bottleneck.** The channel could carry the data without errors, but uncoded BPSK needs about 10 dB before bit errors become smaller than quantization error. ARINC 429 has only parity, which detects some errors and corrects none. An error-correcting code could move the knee most of the way toward the Shannon limit. Modern codes come within about 1–2 dB of it.
3. **Above about 10 dB: the quantizer is the bottleneck.** The error is set by the 8-bit quantizer. Each extra bit per sample would lower the MSE floor by about 6 dB, at the cost of 12.5% more bits on the link.

Framing overhead. Of every 36 bit times (a 32-bit word plus the 4-bit gap), only 16 carry sample bits, which is 44%. The label, SDI, SSM, parity and gap make each word self-identifying on a shared bus, but in Shannon's terms they are overhead. The gap carries no energy, so measured per *sample* bit the energy needed doubles (3 dB). This system therefore needs about 13 dB per information bit, against the -1.59 dB limit.

The bus rate as a bottleneck. A high-speed ARINC 429 bus at 100 kbit/s can send at most $100\,000/36 \approx 2778$ words per second. That is 5556 samples per second with two samples per word. Sending $F_s = 8$ kHz at 8 bits per sample in this framing needs 144 kbit/s, so a real ARINC 429 bus could not carry this stream in real time. Here the protocol's data rate runs out before noise becomes a problem. A 300 Hz signal only needs F_s of about 1–2 kHz, which fits easily.

7 Conclusions

- The full chain works end to end. With a good channel the signal is recovered to within 8-bit quantization error (MSE 6.34×10^{-6}, SQNR 49.0 dB).
- The BER of both the BPSK link and the baseband ARINC bus matches theory. The carrier does not change the error rate at a given E_b/N_0.
- Reconstruction quality is set by channel errors below about 10 dB E_b/N_0 and by the quantizer above it, following $\mathrm{MSE}_q + \frac{4}{3}P_b$.
- Parity lets the receiver drop bad words, but whether that helps depends on what replaces them. Holding the last value made the result worse; interpolation made it better.
- Shannon's theorem allows this rate to be carried error-free from -1.21 dB. The 11 dB gap to the knee is what error-correcting coding could recover. ARINC 429 has none.

8 Limitations

- Bit, word and carrier timing are assumed known at the receiver. A real ARINC 429 receiver recovers timing from the RZ transitions and finds word boundaries from the NULL gap.
- The channel adds only white Gaussian noise. There is no interference, cable dispersion, fading or impulse noise.
- Label 205 and the two-samples-per-word packing were chosen for this lab. A real system uses the label assignments and data formats (BNR or BCD) from the equipment specification.

A Code

The full project, including `main.py` and all modules, is in the GitHub repository. Below are the ARINC 429 framing, line coding, modulation and the receiver chain.

ARINC 429 framing (`functions/arinc429.py`)

```
"""Stage 4a: ARINC 429 framing of the PCM bit stream.

Word layout (ARINC 429 bit numbers, bit 1 is transmitted first):

    bits 1-8 label (sent MSB first, so bit 1 holds the label MSB)
    bits 9-10 SDI (source/destination identifier)
    bits 11-29 data (19 bits, LSB in bit 11)
    bits 30-31 SSM (sign/status matrix)
    bit 32 parity (odd parity over the whole word)
```

```
10
11  In this lab each word carries two 8-bit PCM samples:
12      bits 11-18 sample A, bits 19-26 sample B, bits 27-29 padding (zero).
13
14  Words are handled as an (N, 32) array of bits in transmission order, so
15  column k holds ARINC bit k+1.
16  """
17  import numpy as np
18
19  WORD_BITS = 32
20  SAMPLES_PER_WORD = 2
21
22
23  def _field_lsb_first(values, width):
24      """Integer array -> (N, width) bits, LSB in column 0."""
25      values = np.asarray(values, dtype=np.int64)
26      return ((values[:, None] >> np.arange(width)) & 1).astype(np.uint8)
27
28
29  def _field_value(bits):
30      """(N, width) bits, LSB in column 0 -> integer array."""
31      return bits.astype(np.int64) @ (1 << np.arange(bits.shape[1]))
32
33
34  def pack_pcm_into_words(pcm_bits, n_bits=8, label=0o205, sdi=0, ssm=0b11):
35      """Pack a PCM bit stream (MSB-first samples) into ARINC 429 words.
36
37      Returns (words, n_pad_samples). Odd sample counts are padded with
38      mid-scale code 2**(n_bits-1) so every word carries two samples.
39      """
40      if n_bits != 8:
41          raise ValueError("This framing puts two 8-bit samples in each word.")
42      codes = np.asarray(pcm_bits, dtype=np.int64).reshape(-1, n_bits) @ (
43          1 << np.arange(n_bits - 1, -1, -1))
44      n_pad = (-len(codes)) % SAMPLES_PER_WORD
45      codes = np.concatenate([codes, np.full(n_pad, 2 ** (n_bits - 1))])
46      pairs = codes.reshape(-1, SAMPLES_PER_WORD)
47      n_words = len(pairs)
48
49      words = np.zeros((n_words, WORD_BITS), dtype=np.uint8)
50      label_msb_first = (label >> np.arange(7, -1, -1)) & 1
51      words[:, 0:8] = label_msb_first
52      words[:, 8:10] = _field_lsb_first(np.full(n_words, sdi), 2)
53      words[:, 10:18] = _field_lsb_first(pairs[:, 0], 8)
54      words[:, 18:26] = _field_lsb_first(pairs[:, 1], 8)
55      # bits 27-29 stay zero (padding)
56      words[:, 29:31] = _field_lsb_first(np.full(n_words, ssm), 2)
57      words[:, 31] = 1 - (words[:, :31].sum(axis=1) % 2) # make the word odd
58      return words, n_pad
59
60
61  def parse_words(words):
62      """Decode an (N, 32) bit array back into fields.
63
64      Returns a dict of arrays: label, sdi, codes (N, 2), pad, ssm,
65      parity_ok.
66      """
67      words = np.asarray(words, dtype=np.uint8)
68      label = words[:, 0:8].astype(np.int64) @ (1 << np.arange(7, -1, -1))
69      return {
70          "label": label,
71          "sdi": _field_value(words[:, 8:10]),
72          "codes": np.stack([_field_value(words[:, 10:18]),
73                             _field_value(words[:, 18:26])], axis=1),
```

```
        "pad": _field_value(words[:, 26:29]),
        "ssm": _field_value(words[:, 29:31]),
        "parity_ok": (words.sum(axis=1) % 2) == 1,
    }


def word_to_hex(word_bits):
    """32-bit word (transmission order) -> hex string with bit 32 as the MSB."""
    value = int(np.asarray(word_bits, dtype=np.int64) @ (1 << np.arange(WORD_BITS)))
    return f"0x{value:08X}"


def word_field_table(word_bits):
    """Human-readable breakdown of one word, for the report."""
    f = parse_words(np.asarray(word_bits)[None, :])
    return {
        "label (octal)": f"{int(f['label'][0]):03o}",
        "SDI": int(f["sdi"][0]),
        "sample A code": int(f["codes"][0, 0]),
        "sample B code": int(f["codes"][0, 1]),
        "padding bits": int(f["pad"][0]),
        "SSM": format(int(f["ssm"][0]), "02b"),
        "parity bit": int(word_bits[31]),
        "parity check": "PASS" if f["parity_ok"][0] else "FAIL",
    }
```

Bipolar RZ line code (functions/line_coding.py)

```
"""Stage 4b: ARINC 429 bipolar return-to-zero line code.

ARINC 429 sends each bit as a pulse in the first half of the bit time and
returns to zero (NULL) for the second half:
    1 -> HI (+10 V nominal, differential)
    0 -> LO (-10 V nominal)
Words are separated by at least 4 bit times of NULL, which receivers use
to find the start of the next word.

Here the line code is kept in normalised units (+1 / 0 / -1); multiply by
10 to get the nominal bus voltage.
"""
import numpy as np

ARINC_VOLTS = 10.0


def words_to_levels(words, gap_bits=4):
    """(N, 32) bits -> flat array of bit-slot levels.

    Data bits become +1 / -1, inter-word gap slots become 0.
    Returns (levels, is_data) where is_data marks the data slots.
    """
    words = np.asarray(words, dtype=np.int8)
    n_words = words.shape[0]
    levels = np.zeros((n_words, words.shape[1] + gap_bits))
    levels[:, :words.shape[1]] = 2 * words - 1
    is_data = np.zeros_like(levels, dtype=bool)
    is_data[:, :words.shape[1]] = True
    return levels.ravel(), is_data.ravel()


def bipolar_rz(levels, samples_per_bit=40):
    """Bit-slot levels -> sampled bipolar RZ waveform (pulse in first half)."""
    if samples_per_bit % 2:
```

```
        raise ValueError("samples_per_bit must be even.")
    half = samples_per_bit // 2
    pulse = np.r_[np.ones(half), np.zeros(half)]
    return (np.asarray(levels)[:, None] * pulse).ravel()
```

BPSK modulation and demodulation (functions/modulation.py)

```
"""Stages 5 and 7: BPSK modulation and coherent demodulation.

ARINC 429 itself is a baseband twisted-pair bus with no carrier. To meet
the practical's modulation stage, the ARINC bit stream is modelled as
being relayed over an RF telemetry link (as in flight-test telemetry,
which carries recorded ARINC 429 bus traffic to the ground).

The bipolar RZ waveform multiplies the carrier directly, so a 1 is a
half-bit carrier burst at phase 0, a 0 is a burst at phase pi, and the
NULL halves and inter-word gaps carry no carrier. This is BPSK with RZ
pulse shaping.
"""
import numpy as np


def carrier(n_samples, samples_per_bit=40, cycles_per_bit=4):
    """Unit cosine carrier with an integer number of cycles per bit."""
    n = np.arange(n_samples)
    return np.cos(2 * np.pi * cycles_per_bit * n / samples_per_bit)


def bpsk_modulate(baseband, samples_per_bit=40, cycles_per_bit=4):
    """Multiply the line-coded waveform onto the carrier."""
    c = carrier(len(baseband), samples_per_bit, cycles_per_bit)
    return baseband * c, c


def correlate(received, reference, samples_per_bit=40):
    """Correlation receiver over the active (first) half of each bit slot.

    For BPSK pass the carrier as `reference`; for a plain baseband ARINC
    bus pass an array of ones. Returns one decision statistic per slot.
    """
    half = samples_per_bit // 2
    mixed = (np.asarray(received) * reference).reshape(-1, samples_per_bit)
    return mixed[:, :half].sum(axis=1)


def decide(statistics):
    """Sign decision: positive -> 1, otherwise 0."""
    return (np.asarray(statistics) > 0).astype(np.uint8)
```

Full chain (functions/system.py)

```
"""The full chain, wired together.

Source -> Sampling -> Quantization/PCM -> ARINC 429 framing -> Bipolar RZ
-> BPSK -> AWGN -> Coherent demodulation -> ARINC 429 decoding and checks
-> PCM decoding -> Reconstruction
"""
from dataclasses import dataclass

import numpy as np

```

```
from . import arinc429, channel, line_coding, modulation, pcm, reconstruction, source
from .metrics import mse


@dataclass
class Config:
    fs: float = 8000.0 # sampling rate of the message (Hz)
    fm: float = 300.0 # message frequency (Hz)
    duration: float = 0.05 # message length (s)
    n_bits: int = 8 # PCM bits per sample
    label: int = 0o205 # ARINC label (octal), chosen for this lab
    sdi: int = 0b00
    ssm: int = 0b11 # SSM 11: normal operation (BNR convention)
    gap_bits: int = 4 # NULL bit times between words (ARINC minimum)
    samples_per_bit: int = 40 # simulation samples per ARINC bit time
    cycles_per_bit: int = 4 # carrier cycles per bit time
    bit_rate: float = 100e3 # ARINC 429 high-speed rate (bit/s)


def build_transmitter(cfg=Config()):
    """Run the transmit side once. Returns a dict holding every stage."""
    t, message = source.generate_message(cfg.fs, cfg.fm, cfg.duration)
    quantized, codes, step = pcm.quantize(message, cfg.n_bits)
    pcm_bits = pcm.pcm_encode(codes, cfg.n_bits)
    words, n_pad = arinc429.pack_pcm_into_words(
        pcm_bits, cfg.n_bits, cfg.label, cfg.sdi, cfg.ssm)
    levels, is_data = line_coding.words_to_levels(words, cfg.gap_bits)
    baseband = line_coding.bipolar_rz(levels, cfg.samples_per_bit)
    tx, carrier = modulation.bpsk_modulate(
        baseband, cfg.samples_per_bit, cfg.cycles_per_bit)
    return {
        "cfg": cfg, "t": t, "message": message, "quantized": quantized,
        "codes": codes, "step": step, "pcm_bits": pcm_bits, "words": words,
        "n_pad": n_pad, "levels": levels, "is_data": is_data,
        "baseband": baseband, "carrier": carrier, "tx": tx,
        "n_data_bits": int(is_data.sum()),
    }


def receive(txs, rx, mode="bpsk"):
    """Demodulate, decode ARINC words and rebuild the message samples."""
    cfg = txs["cfg"]
    reference = txs["carrier"] if mode == "bpsk" else np.ones(len(rx))
    stats = modulation.correlate(rx, reference, cfg.samples_per_bit)
    data_stats = stats[txs["is_data"]]
    bits = modulation.decide(data_stats)
    words_rx = bits.reshape(-1, arinc429.WORD_BITS)

    fields = arinc429.parse_words(words_rx)
    word_ok = (fields["parity_ok"]
               & (fields["label"] == cfg.label)
               & (fields["ssm"] == cfg.ssm))

    n = len(txs["message"])
    raw = pcm.code_to_value(fields["codes"].ravel(), cfg.n_bits)
    held = reconstruction.conceal_bad_words(raw, word_ok)
    interp = reconstruction.interpolate_bad_words(raw, word_ok)

    tx_bits = txs["words"].ravel()
    bit_err = bits != tx_bits
    word_err = bit_err.reshape(-1, arinc429.WORD_BITS).any(axis=1)
    return {
        "statistics": stats, "bits": bits, "words": words_rx,
        "fields": fields, "word_ok": word_ok,
```

```
        "samples_raw": raw[:n], "samples_hold": held[:n],
        "samples_interp": interp[:n],
        "bit_errors": int(bit_err.sum()), "n_bits": len(tx_bits),
        "word_errors": int(word_err.sum()),
        "rejected_words": int((~word_ok).sum()),
        "undetected_word_errors": int((word_err & word_ok).sum()),
        "n_words": len(word_ok),
        "mse_raw": mse(txs["message"], raw[:n]),
        "mse_hold": mse(txs["message"], held[:n]),
        "mse_interp": mse(txs["message"], interp[:n]),
    }


def run_link(txs, ebn0_db, rng, mode="bpsk"):
    """One pass through the channel and receiver at a given Eb/N0."""
    tx = txs["tx"] if mode == "bpsk" else txs["baseband"]
    rx, sigma = channel.awgn_ebn0(tx, ebn0_db, txs["n_data_bits"], rng)
    result = receive(txs, rx, mode)
    result.update({"rx": rx, "sigma": sigma, "ebn0_db": ebn0_db})
    return result


def sweep(txs, ebn0_db_values, trials=40, seed=2026, mode="bpsk"):
    """Monte Carlo sweep over Eb/N0. Returns a dict of arrays."""
    rng = np.random.default_rng(seed)
    keys = ["ber", "wer", "reject_rate", "undetected_rate",
            "mse_raw", "mse_hold", "mse_interp"]
    out = {k: [] for k in keys}
    for e in ebn0_db_values:
        acc = {k: 0.0 for k in keys}
        bits = words = 0
        for _ in range(trials):
            r = run_link(txs, e, rng, mode)
            acc["ber"] += r["bit_errors"]
            acc["wer"] += r["word_errors"]
            acc["reject_rate"] += r["rejected_words"]
            acc["undetected_rate"] += r["undetected_word_errors"]
            acc["mse_raw"] += r["mse_raw"]
            acc["mse_hold"] += r["mse_hold"]
            acc["mse_interp"] += r["mse_interp"]
            bits += r["n_bits"]
            words += r["n_words"]
        out["ber"].append(acc["ber"] / bits)
        for k in ["wer", "reject_rate", "undetected_rate"]:
            out[k].append(acc[k] / words)
        for k in ["mse_raw", "mse_hold", "mse_interp"]:
            out[k].append(acc[k] / trials)
    out = {k: np.asarray(v) for k, v in out.items()}
    out["ebn0_db"] = np.asarray(ebn0_db_values, dtype=float)
    out["trials"] = trials
    out["bits_per_point"] = trials * txs["n_data_bits"]
    return out


def quantization_floor(txs):
    """MSE with a perfect channel: only quantization error remains."""
    return mse(txs["message"], txs["quantized"])
```